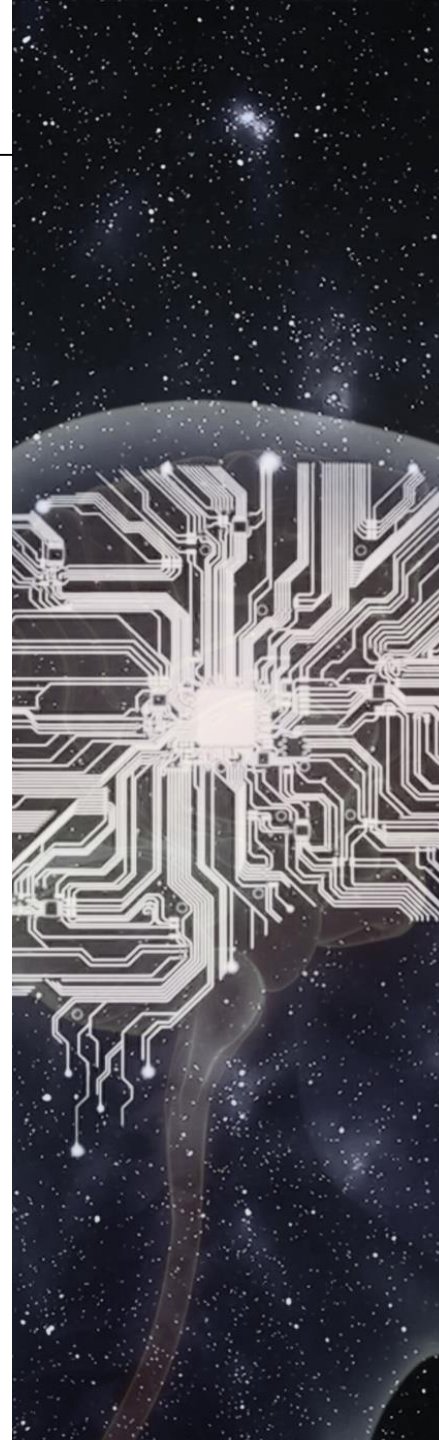
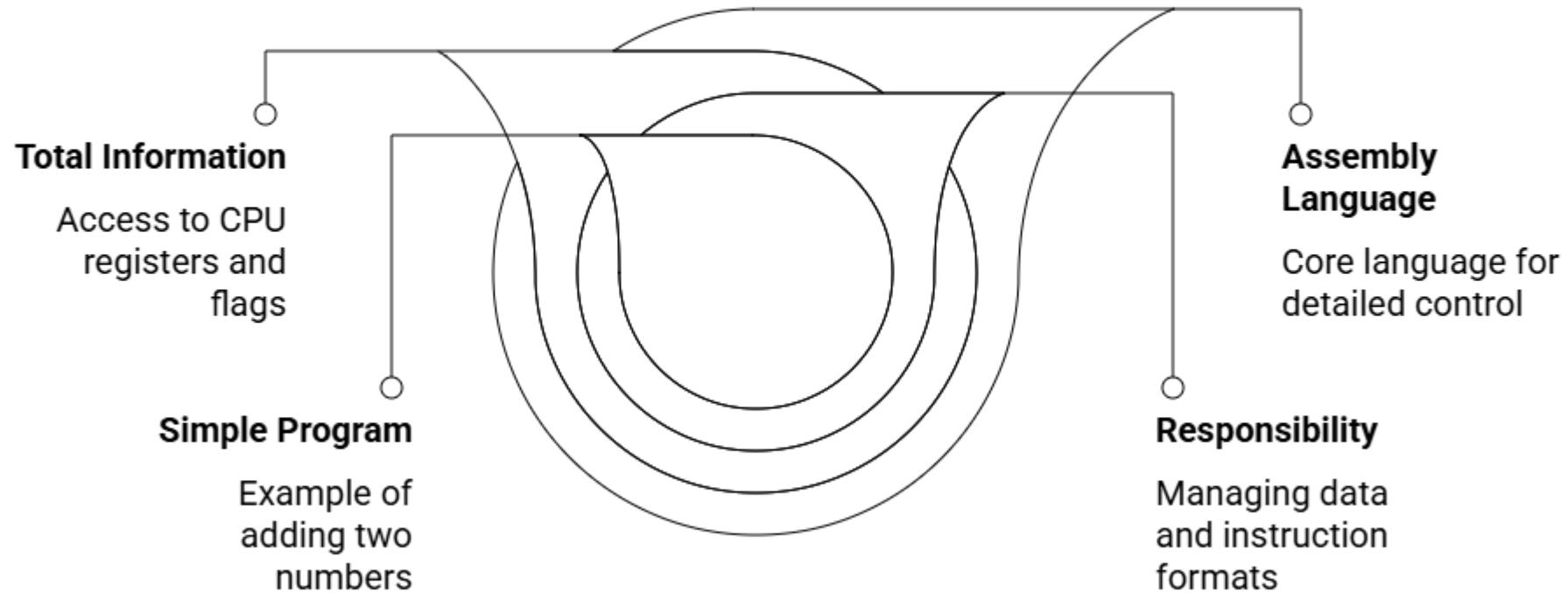


Assembly Programming

3. Assembly Language Fundamentals

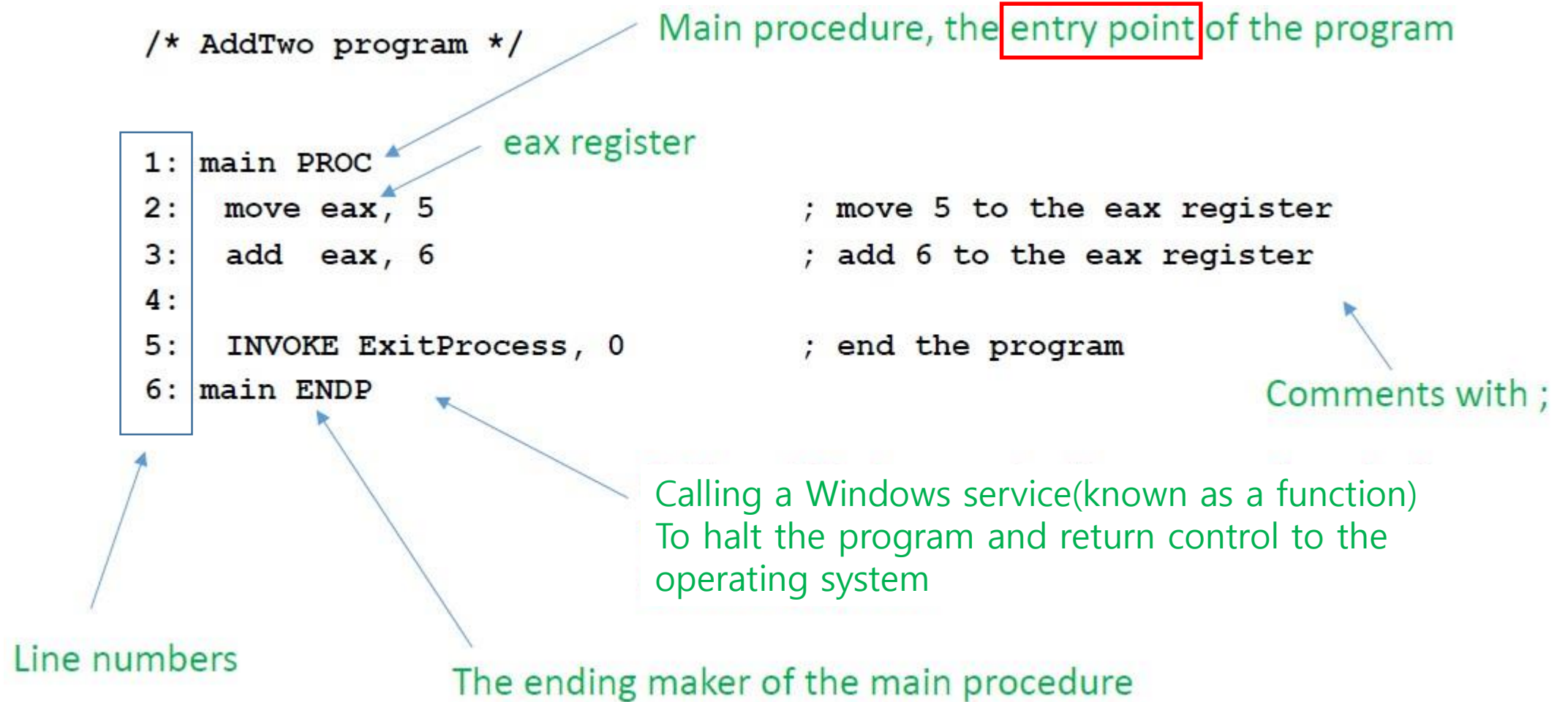


Assembly Language Programming



Made with  Napkin

Basic Language Elements



Basic Language Elements

sum is a variable with a size of 32 bits, using the DRWORD keyword.

Directive

/* Adding a Variable to AddTwo program*/

In Assembly there are a number of size keywords, but they only specify a size.

Data segment

1: .data ; this is the data area
2: sum DRWORD 0 ; create a variable named sum
3:

Code segment

4: .code
5: main PROC
6: move eax, 5 ; move 5 to the eax register
7: add eax, 6 ; add 6 to the eax register
8: mov sum, eax
9:
10: INVOKE ExitProcess, 0 ; end the program
11: main ENDP

We are going to learn language details, how to declare
literals (constants), identifiers, directives, and instructions.

Integer Literals

[{ + | - }] digits [radix]

h	hexadecimal	r	encoded real
q/o	octal	t	decimal (alternate)
d	decimal	y	binary (alternate)
b	binary		

26	; decimal
26d	; decimal
11010011b	; binary
42q	; octal
42o	; octal
1Ah	; hexadecimal
0A3h	; hexadecimal

Constant Integer Expressions

A constant integer expression is a mathematical expression involving integer literals and arithmetic operators.

Table 3-1 Arithmetic Operators.

Operator	Name	Precedence Level
()	Parentheses	1
+, −	Unary plus, minus	2
*, /	Multiply, divide	3
MOD	Modulus	3
+, −	Add, subtract	4

Operator precedence

4 + 5 * 2

Multiply, add

12 - 1 MOD 5

Modulus, subtract

-5 + 2

Unary minus, add

(4 + 2) * 6

Add, multiply

Expression	Value
16 / 5	3
-(3 + 4) * (6 - 1)	-35
-3 + 4 * 6 - 1	20
25 mod 3	1

Real Number Literals

[sign] integer. [integer] [exponent]

sign {+, -}

exponent E[{+, -}] *integer*

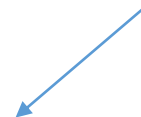
2.

+3.0

-44.2E+05

26.E5

+1.0



0	011	1111	1000	0000	0000	0000	0000	0000	0000
---	-----	------	------	------	------	------	------	------	------

S

E

M

Character Literals

A *character literal* is a single character enclosed in single or double quotes.

ASCII code 'A'
 "d"

String Literals

a sequence of characters (including spaces) enclosed in single or double quotes

'ABC '	"This isn't a test"
'X'	'Say "Good night," Gracie'
"Good night, Gracie"	
'4096 '	

Reserved Words

- Instruction mnemonics, such as MOV, ADD, and MUL
- Register names
- Directives, which tell the assembler how to assemble programs
- Attributes, which provide size and usage information for variables and operands. Examples are BYTE and WORD
- Operators, used in constant expressions
- Predefined symbols, such as @data, which return constant integer values at assembly time

Identifiers

- They may contain between 1 and 247 characters.
- They are not case sensitive.
- The first character must be a letter (A..Z, a..z), underscore (_), @ , ?, or \$. Subsequent characters may also be digits.
- An identifier cannot be the same as an assembler reserved word.

Directives

A *directive* is a command embedded in the source code that is recognized and acted upon by the assembler.

Defining Segments

`.data` `.code` `.stack 100h`

define variables the area of a program containing executable instructions the area of a program holding the runtime stack

Instructions

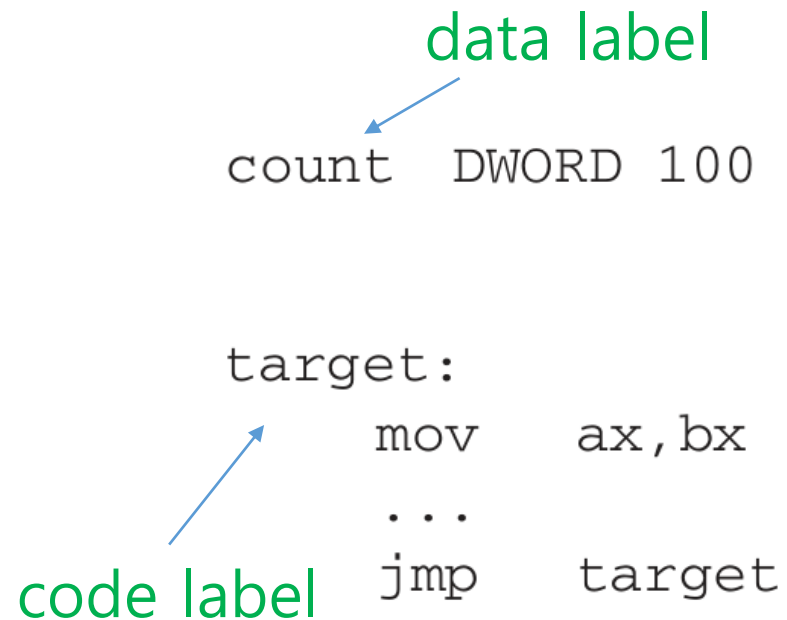
translated by the assembler into machine language bytes

```
[label:] mnemonic [operands] [;comment]
```

- Label (optional)
- Instruction mnemonic (required)
- Operand(s) (usually required)
- Comment (optional)

Label

A label is an identifier that acts as a place marker for instructions and data.



The diagram illustrates two types of labels in assembly code. The first example, 'count', is a data label used in a data definition instruction 'count DWORD 100'. A blue arrow points from the text 'data label' to the 'count' identifier. The second example, 'target:', is a code label used as a jump target in the instruction 'jmp target'. A blue arrow points from the text 'code label' to the 'target:' identifier. Both labels are shown in green text.

```
count  DWORD 100
```



```
target:
    mov    ax, bx
    ...
    jmp    target
```

Instruction Mnemonic

An instruction mnemonic is a short word that identifies an instruction.

Mnemonic	Description
MOV	Move (assign) one value to another
ADD	Add two values
SUB	Subtract one value from another
MUL	Multiply two values
JMP	Jump to a new location
CALL	Call a procedure

Operands

An operand is a value that is used for input or output for an instruction.

Example	Operand Type
96	<i>Integer literal</i>
2 + 4	Integer expression
eax	Register
count	Memory

stc
inc eax
mov count, ebx

; set Carry flag
; add 1 to EAX
; move EBX to count

no operands
one operand
two operands



imul eax, ebx, 5 ; EBX is multiplied by 5, and the product is stored in the EAX register.

three operands

Comments

- Description of the program's purpose
 - Names of persons who created and/or revised the program
 - Program creation and revision dates
 - Technical notes about the program's implementation
-
- Single-line comments, beginning with a semicolon character (;). All characters following the semicolon on the same line are ignored by the assembler.
 - Block comments, beginning with the COMMENT directive and a user-specified symbol. All subsequent lines of text are ignored by the assembler until the same user-specified symbol appears.

```
COMMENT  !  
    This line is a comment.  
    This line is also a comment.  
!
```

```
COMMENT &  
    This line is a comment.  
    This line is also a comment.  
&
```

The NOP (No Operation) Instruction

It takes up 1 byte of program storage and doesn't do any work. It is sometimes used by compilers and assemblers to align code to efficient address boundaries.

00000000	66 8B C3	mov ax,bx	
00000003	90	nop	; align next instruction
00000004	8B D1	mov edx,ecx	

Basic Language Elements

Adding and Subtracting Integers

Getting Started with MASM and Visual Studio 2022

Updated 3/23/2022

This tutorial assumes that you are using either the 7th or 8th Edition of *Assembly Language for x86 Processors*, and you are using Visual Studio 2022.

Before starting the tutorials listed below, please [visit our GitHub project page](#) (surferkip/asmbook) to download the following files:

- Irvine.zip - code examples and required libraries
- Project32_VS2022.zip - sample 32-bit Visual Studio project
- Project64_VS2022.zip - sample 64-bit Visual Studio project

 surferkip Add files via upload	eecd472 · 3 years ago	🕒 9 Commits
 Irvine.zip	Add files via upload	3 years ago
 Project32_VS2019.zip	Add files via upload	3 years ago
 Project32_VS2022.zip	Add files via upload	3 years ago
 Project64_VS2019.zip	Add files via upload	3 years ago
 Project64_VS2022.zip	Add files via upload	3 years ago
 README.md	Update README.md	3 years ago

asmbook / Irvine.zip

 **surferkip** Add files via upload e89abf0 · 3 years ago 🕒 History

Download raw file

Code Blame

Raw   

[View raw](#)

asmbook / Project32_VS2022.zip

 **surferkip** Add files via upload eecd472 · 3 years ago 🕒 History

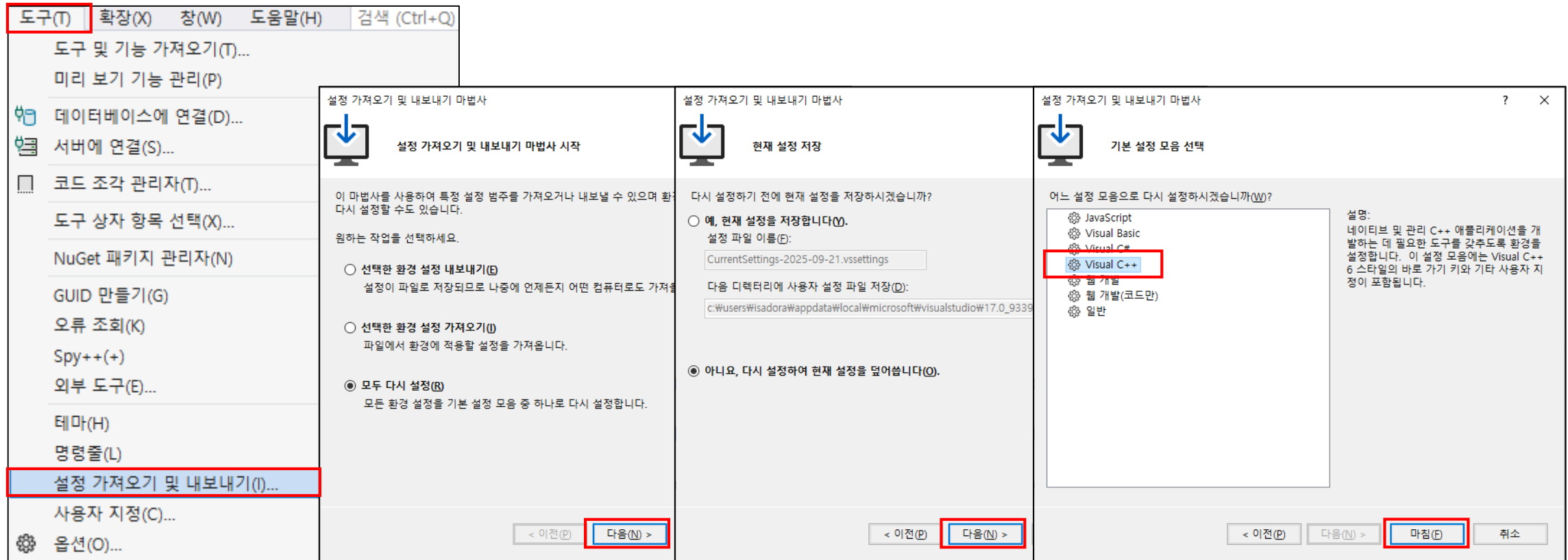
Download raw file

Code Blame

Raw   

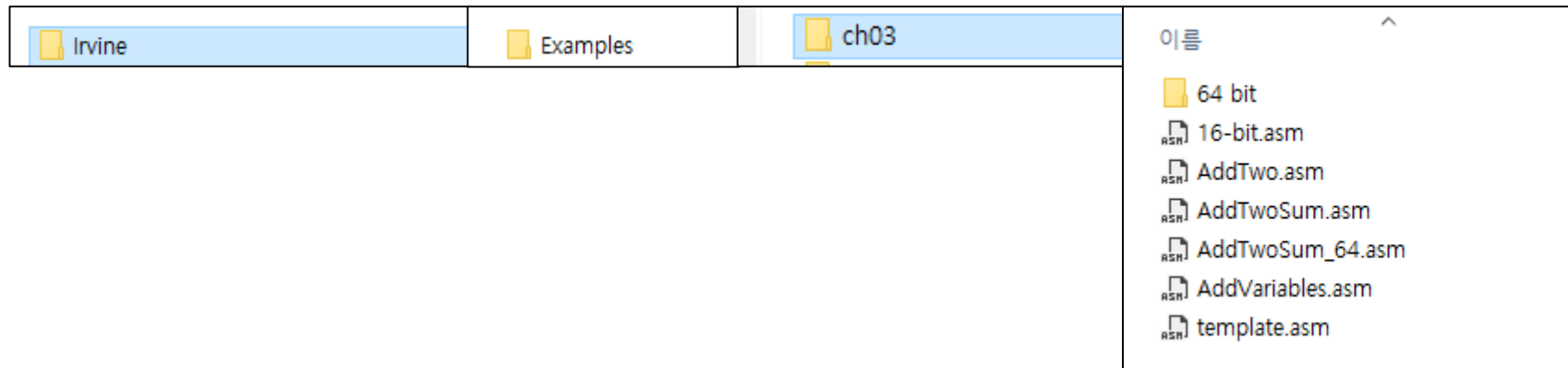
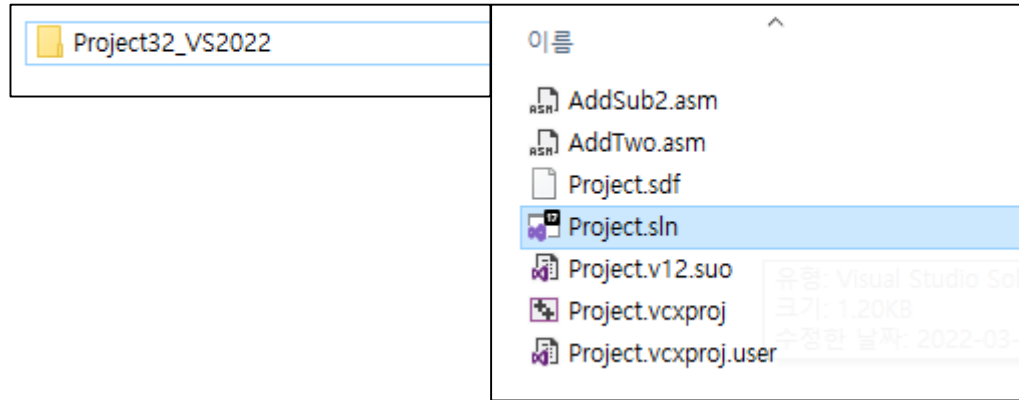
[View raw](#)

Adding and Subtracting Integers



- 솔루션 탐색기 → 프로젝트 우클릭 → 속성 → 링커 → 추가 라이브러리 디렉토리 → C:\Irvine
- 솔루션 탐색기 → 프로젝트 우클릭 → 속성 → 구성 속성 → Microsoft Macro Assembler → General → include Paths : C:\Irvine

Adding and Subtracting Integers



Adding and Subtracting Integers

```
1: ; AddTwo.asm - adds two 32-bit integers
2: ; Chapter 3 example
3:
4: .386    minimum CPU required for this program
5: .model flat,stdcall flat : generate code for protected mode
6: .stack 4096          stdcall : enables the calling of MS Widows functions.
7: ExitProcess PROTO, dwExitCode:DWORD
8: procedure from Irvine32 link lib
9: .code
10: main PROC
11:     mov     eax,5      ; move 5 to the eax register
12:     add     eax,6      ; add 6 to the eax register
13:
14:     INVOKE  ExitProcess,0  INVOKE : called a procedure or function
15: main ENDP
16: END main
```

Adding and Subtracting Integers

The screenshot shows the Microsoft Visual Studio IDE with an assembly file named `AddSub2.asm` open. The code in the editor is as follows:

```
3      ; This program adds and subtracts 32-bit integers
4      ; and stores the sum in a variable.
5
6      INCLUDE Irvine32.inc
7
8      .data
9      val1      dword 10000h
10     val2      dword 40000h
11     val3      dword 20000h
12     finalVal   dword ?
13
14     .code
15     main PROC
16
17         mov eax, val1      ; start with 10000h
18         add eax, val2      ; add 40000h
19         sub eax, val3      ; subtract 20000h
20         mov finalVal, eax  ; store the result (30000h)
21         call DumpRegs     ; display the registers
22
23         exit
24     main ENDP
25     END main
```

On the right side of the IDE, the Solution Explorer shows the project structure:

- 솔루션 탐색기 (Solution Explorer)
- 솔루션 탐색기 검색(Ctrl+;) (Search Solution Explorer)
- 솔루션 'Project' (1 프로젝트의 1) (Solution 'Project' (1 of 1 projects))
- Solution Items
- Project
- 참조 (References)
- 외부 종속성 (External Dependencies)
- AddSub2.asm

In the foreground, a "선택 Microsoft Visual Studio 디버그 콘솔" (Select Microsoft Visual Studio Debug Console) window is open, displaying the following register values and status:

```
EAX=00030000 EBX=00343000 ECX=004010AA EDX=004010AA
ESI=004010AA EDI=004010AA EBP=0019FF80 ESP=0019FF74
EIP=0040367B EFL=00000206 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=1
```

Below the register values, a message is displayed:

```
C:\#Assembly\#Project32_VS2022\#Debug\#Project.exe(프로세스 42612개)이(가) 종료되었습니다(코드: 0개).
이 창을 닫으려면 아무 키나 누르세요...
```

Adding and Subtracting Integers

디버그(D) → 디버깅 시작 F5

```
AddSub2.asm
3      ; This program adds and subtracts 32-bit integers
4      ; and stores the sum in a variable.
5
6      INCLUDE Irvine32.inc
7
8      .data
9      val1      dword 10000h
10     val2      dword 40000h
11     val3      dword 20000h
12     finalVal  dword ?
13
14     .code
15     main PROC
16
17         mov eax, val1      ; start with 10000h
18         add eax, val2      ; add 40000h
19         sub eax, val3      ; subtract 20000h
20         mov finalVal, eax  ; store the result (30000h)
21         call DumpRegs     ; display the registers
22
23         exit
24     main ENDP
25     END main
```

레지스터

EAX	= 0019FFCC
EBX	= 00335000
ECX	= 004010AA
EDX	= 004010AA
ESI	= 004010AA
EDI	= 004010AA
EIP	= 00403660
ESP	= 0019FF74
EBP	= 0019FF80
EFL	= 00000000

0x00406000

177 %

조사식 1

검색(Ctrl+E)

이름	값
eax	17038
val1	65536
val2	262144
val3	131072
finalVal	0

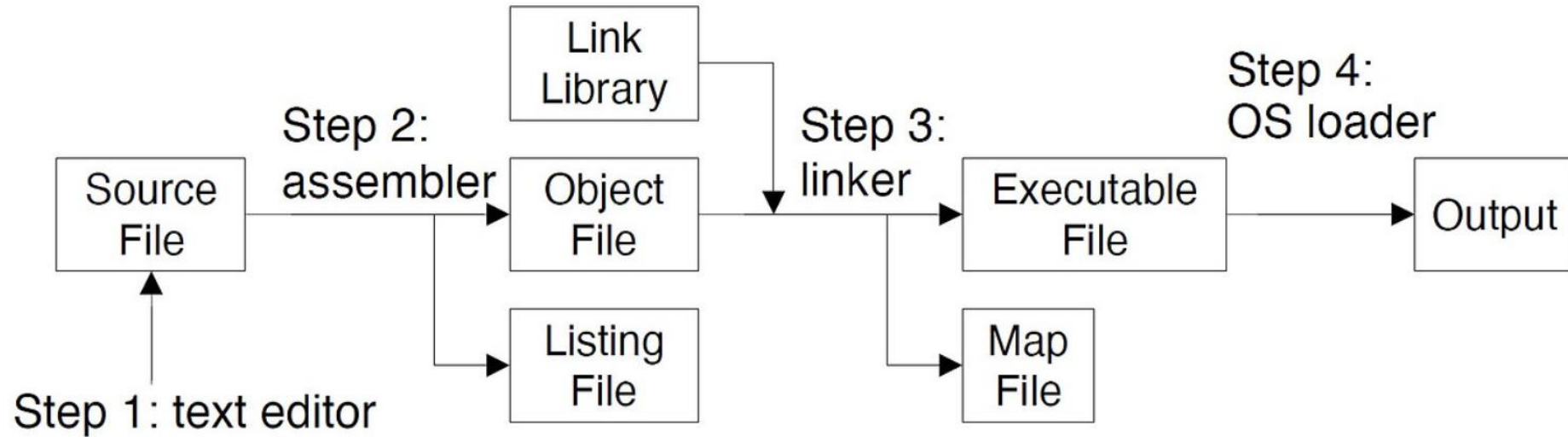
breakpoint

디버그(D)	테스트(S)	분석(N)	도구(T)	확장(X)	창(W)	도움말(H)	검색 (Ctrl+Q)
창(W)					중단점(B)	Ctrl+Alt+B	
그래픽(C)					예외 설정(X)	Ctrl+Alt+E	
계속(C)	F5				출력(O)		
모두 중단(K)	Ctrl+Alt+Break				진단 도구 표시(T)	Ctrl+Alt+F2	
디버깅 중지(E)	Shift+F5				GPU 스레드(U)		
모두 분리(L)					작업(S)	Ctrl+Shift+D, K	
모두 종료(M)					병렬 스택(K)	Ctrl+Shift+D, S	
다시 시작(R)	Ctrl+Shift+F5				병렬 조사식(R)		
코드 변경 내용 적용(A)	Alt+F10				조사식(W)		
성능 프로파일러(F...)	Alt+F2				자동(A)	Ctrl+Alt+V, A	
성능 프로파일러 다시 시작(L)	Shift+Alt+F2				지역(L)	Ctrl+Alt+V, L	
프로세스에 연결(P...)	Ctrl+Alt+P				직접 실행(I)	Ctrl+Alt+I	
기타 디버그 대상(H)					라이브 시각적 트리(V)		
한 단계씩 코드 실행(I)	F11				라이브 속성 탐색기(P)		
프로시저 단위 실행(O)	F10				XAML 라이브 미리보기(R)		
프로시저 나가기(T)	Shift+F11				XAML 바인딩 오류(F)		
간략한 조사식(Q...)	Shift+F9				접근성 검사기(Y)		
중단점 설정/해제(G)	F9				호출 스택(C)	Ctrl+Alt+C	
새 중단점(B)					스레드(H)	Ctrl+Alt+H	
모든 중단점 삭제(A)	Ctrl+Shift+F9				모듈(O)	Ctrl+Alt+U	
모든 중단점 해제(N)					프로세스(P)	Ctrl+Alt+Z	
모든 DataTips 지우기(A)					진단 분석(D)	Ctrl+Shift+Alt+D	
DataTips 내보내기(O...)					메모리(M)		
DataTips 가져오기(I...)					디스어셈블리(D)	Ctrl+Alt+D	
다른 이름으로 덤프 저장(V...)					레지스터(G)	Ctrl+Alt+G	

Assembly programming

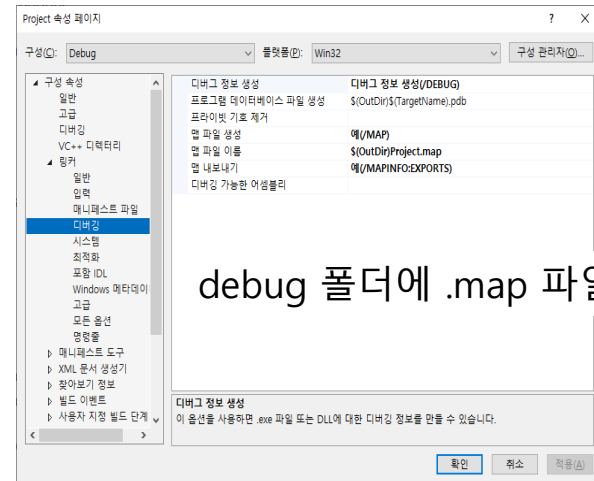
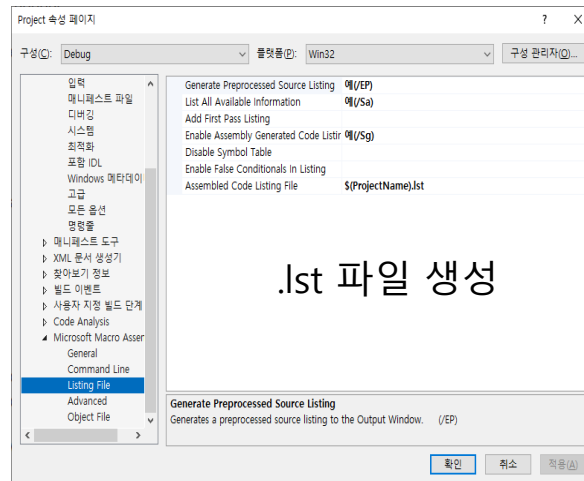
Adding and Subtracting Integers

Assembling, Linking, and Running Programs



`ml /c /coff AddTwo.asm`

`link file1.obj file2.obj file3.obj /SUBSYSTEM:CONSOLE /OUT:Program.exe`



Assembling, Linking, and Running Programs

FIGURE 3–8 Excerpt from the AddTwo source listing file.

```
1:      ; AddTwo.asm - adds two 32-bit integers.
2:      ; Chapter 3 example
3:
4:      .386
5:      .model flat,stdcall
6:      .stack 4096
7:      ExitProcess PROTO,dwExitCode:DWORD
8:
9:      00000000          .code
10:     00000000          main PROC
11:     00000000 B8 00000005      mov  eax,5
12:     00000005 83 C0 06        add  eax,6
13:
14:                                invoke ExitProcess,0
15:     00000008 6A 00          push  +000000000h
16:     0000000A E8 00000000 E    call  ExitProcess
17:     0000000F          main ENDP
18:                                END main
```

Assembling, Linking, and Running Programs

Intrinsic Data Types

The assembler recognizes a basic set of intrinsic data types, which describe types in terms of their size (byte, word, doubleword, and so on), whether they are signed, and whether they are integers or reals.

Data Definition Statement

A data definition statement sets aside storage in memory for a variable, with an optional name.

```
[name] directive initializer [,initializer]...
```

Defining Data

Table 3-2 Intrinsic Data Types.

Type	Usage
BYTE	8-bit unsigned integer. B stands for byte
SBYTE	8-bit signed integer. S stands for signed
WORD	16-bit unsigned integer
SWORD	16-bit signed integer
DWORD	32-bit unsigned integer. D stands for double
SDWORD	32-bit signed integer. SD stands for signed double
FWORD	48-bit integer (Far pointer in protected mode)
QWORD	64-bit integer. Q stands for quad
TBYTE	80-bit (10-byte) integer. T stands for Ten-byte
REAL4	32-bit (4-byte) IEEE short real
REAL8	64-bit (8-byte) IEEE long real
REAL10	80-bit (10-byte) IEEE extended real

Table 3-3 Legacy Data Directives.

Directive	Usage
DB	8-bit integer
DW	16-bit integer
DD	32-bit integer or real
DQ	64-bit integer or real
DT	define 80-bit (10-byte) integer

Defining BYTE and SBYTE Data

- The BYTE (define byte) and SBYTE (define signed byte) directives allocate storage for one or more unsigned or signed values.
- A question mark (?) initializer leaves the variable uninitialized, implying that it will be assigned a value at runtime
- The DB directive can also define an 8-bit variable, signed or unsigned

```
value1 BYTE 'A'           ; character literal
value2 BYTE 0              ; smallest unsigned byte
value3 BYTE 255            ; largest unsigned byte
value4 SBYTE -128          ; smallest signed byte
value5 SBYTE +127          ; largest signed byte
value6 BYTE ?
```

```
val1 DB 255                ; unsigned byte
val2 DB -128               ; signed byte
```

Multiple Initializers

If multiple initializers are used in the same data definition, its label refers only to the offset of the first initializer.

```
list BYTE 10,20,30,40
```

FIGURE 3-11 Memory layout of a byte sequence.

Offset	Value
0000:	10
0001:	20
0002:	30
0003:	40

```
list BYTE 10,20,30,40
      BYTE 50,60,70,80
      BYTE 81,82,83,84
```

```
list1 BYTE 10, 32, 41h, 00100010b
list2 BYTE 0Ah, 20h, 'A', 22h
```

Defining Strings

To define a string of characters, enclose them in single or double quotation marks. The most common type of string ends with a null byte (containing 0). Called a null-terminated string, strings of this type are used in many programming languages

```
greeting1 BYTE "Good afternoon",0  
greeting2 BYTE 'Good night',0
```

```
greeting1 BYTE 'G','o','o','d'....etc.
```

```
greeting1 BYTE "Welcome to the Encryption Demo program "  
    BYTE "created by Kip Irvine.",0dh,0ah  
    BYTE "If you wish to modify this program, please "  
    BYTE "send me a copy.",0dh,0ah,0
```

DUP Operator

The DUP operator allocates storage for multiple data items, using a integer expression as a counter.

```
BYTE 20 DUP(0)           ; 20 bytes, all equal to zero
BYTE 20 DUP(?)           ; 20 bytes, uninitialized
BYTE 4  DUP("STACK")     ; 20 bytes: "STACKSTACKSTACKSTACK"
```

Defining WORD and SWORD Data

The WORD (define word) and SWORD (define signed word) directives create storage for one or more 16-bit integers

```
myList WORD 1,2,3,4,5
```

	Offset	Value
word1		
word2	0000:	1
word3	0002:	2
val1	0004:	3
val2	0006:	4
array	0008:	5

```
word1 WORD 65535           ; largest unsigned value
word2 SWORD -32768         ; smallest signed value
word3 WORD ?              ; uninitialized, unsigned

val1 DW 65535             ; unsigned
val2 DW -32768            ; signed

array WORD 5 DUP(?)       ; 5 values, uninitialized
```

Defining DWORD and SDWORD Data

The DWORD directive (define doubleword) and SDWORD directive (define signed double word) allocate storage for one or more 32-bit integers

```
val1 DWORD    12345678h           ; unsigned
val2 SDWORD   -2147483648         ; signed
val3 DWORD    20 DUP(?)           ; unsigned array

val1 DD 12345678h                 ; unsigned
val2 DD -2147483648               ; signed

pVal DWORD val3

myList DWORD 1,2,3,4,5
```

Offset	Value
0000:	1
0004:	2
0008:	3
000C:	4
0010:	5

Defining Packed BCD (TBYTE) Data

Intel stores a packed binary coded decimal (BCD) integers in a 10-byte package. Each byte (except the highest) contains two decimal digits.

Decimal Value	Storage Bytes
+1234	34 12 00 00 00 00 00 00 00 00
−1234	34 12 00 00 00 00 00 00 00 80

```
intVal TBYTE 80000000000000001234h      ; valid
intVal TBYTE -1234                        ; invalid
```

Defining Floating-Point Types

REAL4 defines a 4-byte single-precision floating-point variable. REAL8 defines an 8-byte double precision value, and REAL10 defines a 10-byte extended-precision value.

```
rVal1      REAL4  -1.2
rVal2      REAL8   3.2E-260
rVal3      REAL10  4.6E+4096
ShortArray REAL4   20 DUP(0.0)

rVal1 DD  -1.2                ; short real
rVal2 DQ   3.2E-260           ; long real
rVal3 DT   4.6E+4096          ; extended-precision real
```

Table 3-4 Standard Real Number Types.

Data Type	Significant Digits	Approximate Range
Short real	6	1.18×10^{-38} to 3.40×10^{38}
Long real	15	2.23×10^{-308} to 1.79×10^{308}
Extended-precision real	19	3.37×10^{-4932} to 1.18×10^{4932}

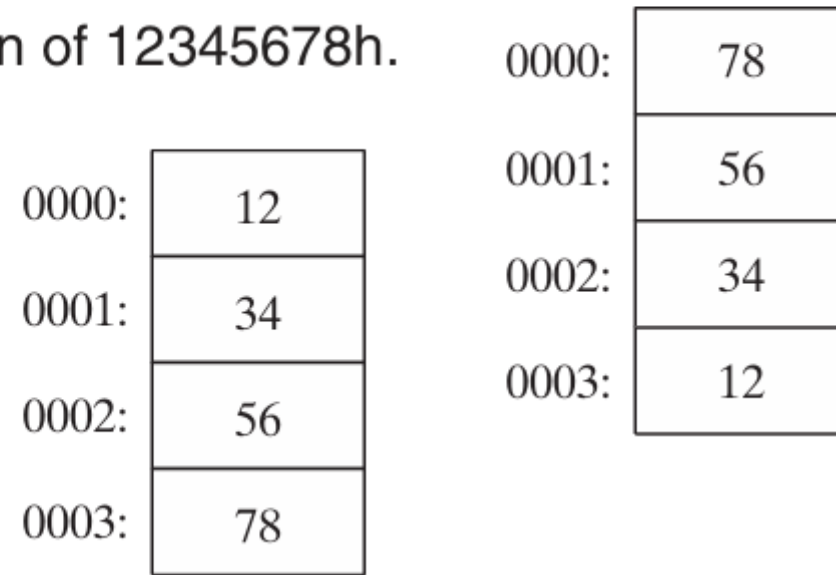
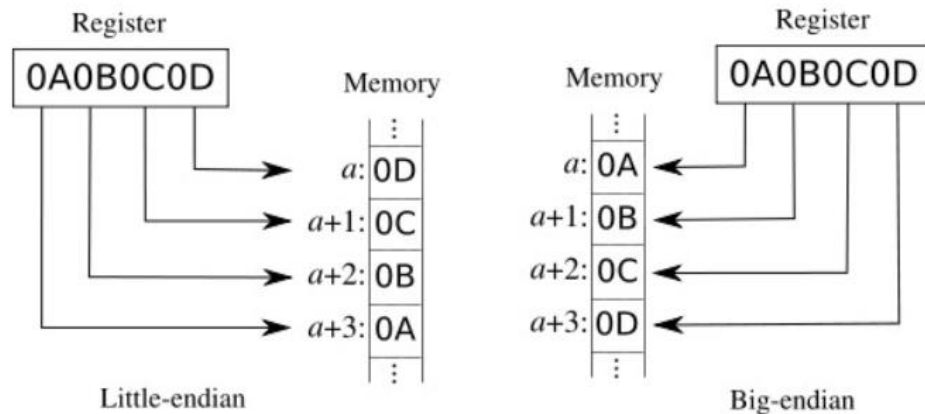
Little-Endian Order

x86 processors store and retrieve data from memory using little-endian order (low to high).

FIGURE 3-14 Little-endian representation of 12345678h.

FIGURE 3-15 Big-endian representation of 12345678h.

리틀 엔디안	빅 엔디안
Intel 계열의 CPU	AMD 계열의 CPU
연산에 최적화	네트워크에 최적화
최하위 바이트부터 저장	최상위 바이트부터 저장



Declaring Uninitialized Data

The `.DATA?` directive declares uninitialized data. When defining a large block of uninitialized data, the `.DATA?` directive reduces the size of a compiled program.

```
.data
smallArray DWORD 10 DUP(0)      ; 40 bytes
.data?
bigArray DWORD 5000 DUP(?)      ; 20,000 bytes, not initialized
```

```
.data
smallArray DWORD 10 DUP(0)      ; 40 bytes
bigArray DWORD 5000 DUP(?)      ; 20,000 bytes
```

Mixing Code and Data

The assembler lets you switch back and forth between code and data in your programs. You might, for example, want to declare a variable used only within a localized area of a program.

```
.code  
mov  eax,ebx  
.data  
temp DWORD ?  
.code  
mov  temp,eax
```

Defining Data

Equal-Sign Directive

The equal-sign directive associates a symbol name with an integer expression

```
name = expression
```

```
COUNT = 500  
mov eax, COUNT
```

Current Location Counter

```
selfPtr DWORD $
```

Keyboard Definitions

```
Esc_key = 27
```

Using the DUP Operator

```
array dword COUNT DUP(0)
```

Redefinitions

```
COUNT = 5  
mov al,COUNT ; AL = 5  
COUNT = 10  
mov al,COUNT ; AL = 10  
COUNT = 100  
mov al,COUNT ; AL = 100
```

Calculating the Sizes of Arrays and Strings

```
list BYTE 10,20,30,40
ListSize = 4
```

```
list BYTE 10,20,30,40
ListSize = ($ - list)
```

```
list BYTE 10,20,30,40
var2 BYTE 20 DUP(?)
ListSize = ($ - list)
```

```
myString BYTE "This is a long string, containing"
          BYTE "any number of characters"
myString_len = ($ - myString)
```

```
list WORD 1000h,2000h,3000h,4000h
ListSize = ($ - list) / 2
```

```
list DWORD 10000000h,20000000h,30000000h,40000000h
ListSize = ($ - list) / 4
```

Symbolic Constants

EQU Directive

The EQU directive associates a symbolic name with an integer expression or some arbitrary text.

```
name EQU expression
```

```
name EQU symbol
```

```
name EQU <text>
```

```
PI EQU <3.1416>      pressKey EQU <"Press any key to continue...",0>
```

```
.
```

```
.
```

```
.data
```

```
prompt    BYTE    pressKey
```

```
matrix1    EQU    10 * 10
```

```
matrix2    EQU    <10 * 10>
```

```
.data
```

```
M1 WORD matrix1
```

```
M1 WORD 100
```

```
M2 WORD matrix2
```

```
M2 WORD 10 * 10
```

TEXTEQU Directive

The TEXTEQU directive, similar to EQU, creates what is known as a text macro.

```
name TEXTEQU <text>
name TEXTEQU textmacro
name TEXTEQU %constExpr
```

```
continueMsg TEXTEQU <"Do you wish to continue (Y/N)?">
.data
prompt1 BYTE continueMsg
```

```
rowSize = 5
count    TEXTEQU    %(rowSize * 2)
move     TEXTEQU    <mov>
setupAL  TEXTEQU    <move al,count>           mov al,10
```

Symbolic Constants

3 Assembly Language Fundamentals 53

3.1 Basic Language Elements 54

- 3.1.1 First Assembly Language Program 54
- 3.1.2 Integer Literals 55
- 3.1.3 Constant Integer Expressions 56
- 3.1.4 Real Number Literals 57
- 3.1.5 Character Literals 57
- 3.1.6 String Literals 58
- 3.1.7 Reserved Words 58
- 3.1.8 Identifiers 58
- 3.1.9 Directives 59
- 3.1.10 Instructions 60
- 3.1.11 Section Review 63

3.2 Example: Adding and Subtracting Integers 63

- 3.2.1 The *AddTwo* Program 63
- 3.2.2 Running and Debugging the *AddTwo* Program 65
- 3.2.3 Program Template 70
- 3.2.4 Section Review 70

3.3 Assembling, Linking, and Running Programs 71

- 3.3.1 The Assemble-Link-Execute Cycle 71
- 3.3.2 Listing File 71
- 3.3.3 Section Review 73

3.4 Defining Data 74

- 3.4.1 Intrinsic Data Types 74
- 3.4.2 Data Definition Statement 74
- 3.4.3 Adding a Variable to the *AddTwo* Program 75
- 3.4.4 Defining BYTE and SBYTE Data 76
- 3.4.5 Defining WORD and SWORD Data 78
- 3.4.6 Defining DWORD and SDWORD Data 79
- 3.4.7 Defining QWORD Data 79
- 3.4.8 Defining Packed BCD (TBYTE) Data 80
- 3.4.9 Defining Floating-Point Types 81
- 3.4.10 A Program That Adds Variables 81
- 3.4.11 Little-Endian Order 82
- 3.4.12 Declaring Uninitialized Data 83
- 3.4.13 Section Review 83

3.5 Symbolic Constants 84

- 3.5.1 Equal-Sign Directive 84
- 3.5.2 Calculating the Sizes of Arrays and Strings 85
- 3.5.3 EQU Directive 86
- 3.5.4 TEXTEQU Directive 87
- 3.5.5 Section Review 88

3.6 64-Bit Programming 88

3.7 Chapter Summary 90

3.8 Key Terms 91

- 3.8.1 Terms 91
- 3.8.2 Instructions, Operators, and Directives 92

3.9 Review Questions and Exercises 92

- 3.9.1 Short Answer 92
- 3.9.2 Algorithm Workbench 93

3.10 Programming Exercises 94

reference

<https://www.asmirvine.com/>

<https://intelligentcm.tistory.com/category/Assembly>

Review questions and Exercises

Github upload

Presentation subject

- Flag register
- Segment register